Replit Approach

Step 1: Account Creation & Initial Setup on Replit

1. **Access Replit:** Open a web browser and navigate to `replit.com`.
2. **Initiate Registration:** Click **Create Account** in the upper-right corner of the interface.
3. **Authenticate:** Select **Continue with GitHub** to authenticate using your GitHub credentials.
4. **Verify Email:** Check your inbox and follow the verification link sent by Replit to validate your account.
5. **Complete Onboarding:** Answer the required onboarding prompts, then select **Continue with Starter** on the plan selection screen.

Step 2: Generate the Master Mega-Prompt using Gemini

Prepare the Master Prompt: Prior to environment setup in Replit, use an LLM interface (such as Gemini or Claude) alongside the provided Meta-Prompt to draft a comprehensive application specification.

1. **Access Gemini:** Open gemini.google.com in a new tab.
2. **Generate Specification:** Paste the Meta-Prompt provided below into the Gemini console to generate the complete system requirements and prompt payload:

```
You are an expert Prompt Engineer, AI Systems Architect, and Principal
Full-Stack Engineer.

Your task is to take the raw assignment details for an "AI-Powered Agriculture
Crop Advisory Assistant" and write an exhaustive, production-grade, end-to-end
"Master Prompt" (Mega Prompt) that a developer can paste directly into an AI
coding agent (like Cursor, Claude Code, Replit Agent, or Bolt) to build the
complete application without leaving placeholders or skipping steps.

The output Mega Prompt MUST strictly mirror the exact layout, tone, section
hierarchy, markdown formatting, technical depth, and structural conventions of
the reference prompt provided below.

---

### REFERENCE PROMPT STRUCTURE TO COPY EXACTLY:

1. Header & System Persona Role
2. Core Mission Directive
3. Product Goal
4. Mandatory Features
5. Technology Requirements (React.js, Node.js + Express.js, Replit Postgres,
@google/genai SDK, Zod, Tailwind CSS)
6. Application Pages (Routes)
7. User Flow
```

```
8. Target Domains & Categorization
9. Form & Advisory Configuration
10. Database Schema (Production SQL for PostgreSQL)
11. Row Level Security (RLS) / Data Isolation Rules
12. Backend API Routes
13. Gemini SDK Setup & Server-Side Security
14. AI System Prompt
15. Detailed AI Prompts (with required JSON Schemas)
16. Zod Validation Requirements
17. Frontend Components List
18. Security Requirements
19. Environment Variables Template
20. Suggested Folder Structure
21. Implementation Phases
22. Acceptance Criteria
23. Final Instruction to Coding Agent

---

### RAW ASSIGNMENT INPUT DATA:
<Your problem statement, solution and relevant information>

Generate the complete Master Prompt now.
```

3. Copy the complete **Mega Prompt** generated by Gemini.

Step 3: Build in Replit Agent & Configure Replit Database

1. **Input Prompt Payload:** In your Replit workspace, paste the generated Mega-Prompt into the Replit Agent input panel.
2. **Trigger Generation:** Click **Build** to initiate the automated generation process.
3. **Configure the Database:** When prompted for database preferences or Supabase credentials, specify: *"Use the database provided by Replit instead of Supabase."*
4. **Automated Scaffolding:** Replit Agent will automatically generate the directory architecture, resolve dependencies, generate the frontend and backend source code, and provision the local PostgreSQL schema.

Step 4: Sync & Push Your Repository to GitHub

1. **Open Git Tools:** Click **Tools** in the Replit navigation bar and select **Git**.
2. **Authorize GitHub Integration:** Open **Settings** under the Git panel and log in to your GitHub account to grant the required OAuth permissions.

3. **Initialize Remote Repository:** Click **Create Remote**, input your designated repository name (e.g., `CropPilot-AI`), and click **Create Repository on GitHub**.
4. **Return to Git Panel:** Click the **Back** arrow to return to the primary Git view.
5. **Enable Credential Forwarding:** Toggle/check the **Pass GitHub Credentials** option.
6. **Push Codebase:** Click **Push branch as 'origin/main'** to publish your codebase directly to your remote GitHub repository.

Step 5: Add API Keys in Replit Secrets

1. **Access Secrets Manager:** Navigate to **Tools** in the Replit interface and select **Secrets**.
2. **Create New Secret:** Click **New Secret** to initiate adding an environment variable.
3. **Configure Gemini API Key:**
    - **Key:** `GEMINI_API_KEY`
    - **Value:** Your Google AI Studio API Key.
    - *To generate your key:*
        1. Navigate to aistudio.google.com and log in with your Google account.
        2. Click the **Get API key** icon located in the lower-left sidebar.
        3. Click **Create API key**, assign a name to your key, and click **Create Key**.
        4. Copy the generated key immediately and paste it into the Replit Secrets value field.

4. **Configure JWT Authentication Secret:**

    - **Key:** JWT_SECRET
    - **Value:** A secure random secret string used for backend authentication — you can use value like your name along with your project title.
    - **Recommended:** Generate a cryptographically secure key locally using a command like `openssl rand -hex 32` (or `openssl rand -base64 32`) in Git Bash terminal, so the secret is never transmitted over the network.

5. **Save Variables:** Click **Add Secret** after entering each key-value pair to commit them to your workspace environment.

Step 6: Feature Verification & AI Debugging (Replit Preview)

1. **Launch Application Preview:** Open the **Preview** tab (**Webview**) within the Replit workspace to begin manual testing.
2. **Execute Feature Verification Checklist:**
    - **Authentication:** Test registering a new farmer account, logging out, and logging back into the platform.
    - **Farm Management:** Create a farm entry, add a field, and log a crop record.

    - **AI Advisory Engine:** Submit a crop diagnosis query (e.g., *"Yellow leaf spots on flowering tomatoes"*). Verify that Gemini returns a condition analysis, action plan, and necessary disclaimers.
    - **Database Persistence:** Refresh the browser page and confirm that advisory records and farm data persist correctly.
3. **Resolve Issues via Follow-Up Prompts:**
    - If an application error occurs or a UI element fails, copy the exact error output or describe the issue to Replit Agent in the chat interface.
    - *Example Bug Fix Prompt:*
      "I encountered an error when submitting the AI crop advisor form: '[Insert Error Message]'. Please resolve the backend route / database query so the form successfully saves the advisory and returns Gemini's output."
    - Replit Agent will analyze the codebase, refactor the faulty logic, and re-test the application automatically.
4. **Sync Updates to GitHub:**
    - After Replit Agent resolves an issue or modifies source files, sync the updated code to your remote repository prior to deployment.
    - Navigate to **Tools** → **Git** to review modified files.
    - Enter a concise commit message (e.g., `"Fixed AI crop advisor form error"`).
    - Click **Stage and commit all changes**.
    - Click **Push** (or **Push branch as 'origin/main'**) to push the stable codebase to GitHub.

Step 7: Publish & View Your Live Application

1. **Initiate Deployment:** Click **Publish** in the top-right header of the Replit interface.
2. **Configure Settings:** Follow the on-screen configuration workflow to set up your deployment parameters.
3. **Deploy Application:** Click **Publish** to trigger Replit's automated compilation, build, and hosting pipeline (this may take a few minutes).
4. **Access Live Application:** Upon build completion, Replit will output your public application URL (e.g., `https://croppilot-ai.replit.app`). Click the link to verify your live, fully deployed production app.

Antigravity Approach

Step 1: Download & Install Google Antigravity IDE

1. **Access the Portal:** Open a web browser and navigate to antigravity.google
2. **Locate Installer:** Click on Download for Windows option and scroll down to the Antigravity IDE section and download the appropriate installer for your operating system (Windows, macOS, or Linux).
3. **Install and Launch:** Run the downloaded installer file and launch **Google Antigravity IDE**.
4. **Authenticate:** Sign in using your Google account to complete the login process.
5. **Complete Setup Wizard:** Follow the initial setup prompts to select your preferred UI theme and grant standard terminal execution permissions.

Step 2: Configure Supabase Cloud Project & Retrieve API Credentials

1. **Access Supabase:** Navigate to supabase.com and click **Sign Up** (or authenticate using your GitHub account).
2. **Create Organization & Initialize Project:**
    - Click **Create Organization** to establish your workspace.
    - Enter a designated **Project Name** and set a secure database password.
    - Select your preferred deployment **Region** and ensure **Enable automatic RLS** (Row Level Security) is checked.
    - Click **Create New Project** to initiate database provisioning.
3. **Store Project URL:** Once generated, copy the **Project URL** from the project dashboard home page and store it securely for later deployment into your .env configuration.
4. **Retrieve API Keys:**
    - Navigate to **Project Settings** (gear icon in the left sidebar) $\rightarrow$ **API Keys**.
    - Under the **Legacy anon, service_role API keys** section, copy and store the following credentials for later deployment into your .env configuration:
        - **anon public key** (*Client-side publishable key*)
        - **service_role secret key** (*Backend administrative key*)

Step 3: Generate Gemini API Key

1. **Access Google AI Studio:** Navigate to `aistudio.google.com` and sign in with your Google account credentials.
2. **Initiate Key Creation:** Click **Get API key** located in the bottom-left navigation sidebar.

3. **Provision API Key:** Click **Create API key**, select or create your Google Cloud project, assign a key label, and click **Create Key**.
4. **Secure API Key:** Copy the generated API key string (e.g., `AIzaSy...`) and store it securely for environment configuration.

Step 4: Generate the Master Mega-Prompt using Gemini

Prepare the Master Prompt: Prior to project initialization in Google Antigravity IDE, use an LLM interface (such as Gemini, Claude, etc.,) alongside the provided Meta-Prompt to generate a comprehensive application specification.

1. **Access Gemini:** Open gemini.google.com in a new tab.
2. **Generate Specification:** Copy and paste the Meta-Prompt below into the Gemini console to build your complete system prompt and technical requirements payload:

```
You are an expert Prompt Engineer, AI Systems Architect, and Principal
Full-Stack Engineer.
Your task is to take the raw assignment details for an "AI-Powered
Agriculture Crop Advisory Assistant" and write an exhaustive,
production-grade, end-to-end "Master Prompt" (Mega Prompt) that a developer
can paste directly into an AI coding agent (like Cursor, Claude Code,
Replit Agent, or Antigravity Agent) to build the complete application
without leaving placeholders or skipping steps.
The output Mega Prompt MUST strictly mirror the exact layout, tone, section
hierarchy, markdown formatting, technical depth, and structural conventions
of the reference prompt provided below.

### REFERENCE PROMPT STRUCTURE TO COPY EXACTLY:

1. Header & System Persona Role
2. Core Mission Directive
3. Product Goal
4. Mandatory Features
5. Technology Requirements (React.js, Node.js + Express.js, Supabase
PostgreSQL & Auth, @google/genai SDK, Zod, Tailwind CSS)
6. Application Pages (Routes)
7. User Flow
8. Target Domains & Categorization
9. Form & Advisory Configuration
10. Database Schema (Production SQL for PostgreSQL)
11. Row Level Security (RLS) / Data Isolation Rules
```

```
12. Backend API Routes
13. Gemini SDK Setup & Server-Side Security
14. AI System Prompt
15. Detailed AI Prompts (with required JSON Schemas)
16. Zod Validation Requirements
17. Frontend Components List
18. Security Requirements
19. Environment Variables Template
20. Suggested Folder Structure
21. Implementation Phases
22. Acceptance Criteria
23. Final Instruction to Coding Agent

### RAW ASSIGNMENT INPUT DATA:
<Your problem statement, solution and relevant information>

Generate the complete Master Prompt now.
```

3. Copy the complete **Mega Prompt** generated by Gemini.

Step 5: Build in Antigravity Agent & Execute Database Migrations

1. **Initialize Local Workspace:** Create an empty project directory on your local system (e.g., CropPilotAI).
2. **Open Project in IDE:** Launch **Google Antigravity IDE**, select **File** → **Open Folder**, and choose your newly created workspace directory.
3. **Input Mega-Prompt Payload:** Paste your generated Master Mega-Prompt into the Antigravity Agent chat panel on the right sidebar.
4. **Append Supabase Integration Rule:** Append the following precise directive to the end of your prompt:
   "We are using Supabase Cloud PostgreSQL. Create a /supabase/migrations/001_initial_schema.sql file containing all table creation scripts, Row Level Security (RLS) policies, and seed data. Also configure a script/migration runner or Node.js migration script to apply these tables directly to Supabase using the service role key."
5. **Trigger Generation:** Press **Enter** or click **Send**. Antigravity Agent will generate the backend architecture (/server), React frontend (/client), install required npm dependencies, and create the SQL migration assets.

6. **Create Environment Files:** In the Antigravity file explorer, navigate to the `/server` directory and create (or open) a `.env` file. If client-side public keys are required, create a corresponding `.env` file in the `/client` directory.
7. **Populate Environment Secrets:** Add the required key-value pairs using the following schema:

```
# Backend server/.env
PORT=5000
NODE_ENV=development
CLIENT_URL=http://localhost:5173
GEMINI_API_KEY=your_gemini_api_key_here
SUPABASE_URL=https://your-supabase-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key_here
```

```
# Frontend client/.env
VITE_SUPABASE_URL=your_supabase_project_url_from_step_2
VITE_SUPABASE_ANON_KEY=your_supabase_anon_key_from_step_2
VITE_API_BASE_URL=http://localhost:5000
```

8. **Execute Database Migrations:**
    - **Option A (Automated via Agent):** Prompt Antigravity Agent: *"Run the database migration script to execute `001_initial_schema.sql` against our Supabase project."*
    - **Option B (Manual via Supabase Console):** Open your Supabase Dashboard, navigate to **SQL Editor** in the left navigation panel, paste the contents of `/supabase/migrations/001_initial_schema.sql`, and click **Run**.
9. **Verify Database Schema:** Navigate to the **Table Editor** panel (located directly above the SQL Editor in the Supabase sidebar) to inspect and verify all newly created database tables and schema relationships.

Step 6: Feature Verification & AI Debugging (Antigravity Embedded Browser/Preview)

1. **Launch Application Services:**
    - **Option A (Automated via Agent):** Prompt Antigravity Agent in chat: *"Start both the Express backend server and the React frontend dev server."*
    - **Option B (Manual via Terminal):** Open terminal instances and run npm run dev within both the ./server and ./client directories.

2. **Execute Feature Verification Checklist:**
    - **Authentication:** Register a new farmer account, test session termination (logout), and log back in. Verify that the newly created record appears under **Supabase Dashboard** $\rightarrow$ **Authentication**.
    - **Farm Management:** Add a farm entity, configure a field, and register a crop record. Inspect **Supabase Dashboard** $\rightarrow$ **Table Editor** to ensure database persistence across tables.
    - **AI Advisory Engine:** Dispatch a crop diagnosis request (e.g., *"Yellow leaf spots on flowering tomatoes"*). Verify that Gemini synthesizes a condition analysis, actionable remediation plan, and necessary disclaimers.
    - **Database History:** Reload the web client and confirm that historical advisory records persist and render correctly.
3. **Resolve Runtime Issues via Agent Prompts:**
    - If runtime exceptions or UI execution failures occur, copy the terminal stack trace or error payload directly into the Antigravity Agent chat interface.
    - *Example Bug Fix Prompt:*
      "I encountered an error when submitting the AI crop advisor form: '[Insert Error Message]'. Please resolve the backend route / database query so the form saves the advisory to Supabase correctly."
    - Antigravity Agent will analyze the repository stack, refactor faulty modules, and verify operational execution.

Step 6.5: Install Git & Configure Git Identity

1. Install Git (If not already installed on your system)

Windows:

- Install Git & Configure Git Identity

macOS:

- Open Terminal and run the Xcode Command Line Tools installer:

```Bash
    xcode-select --install
```

2. Verify Git Installation

- Open your terminal (or the integrated terminal in Antigravity IDE using Ctrl + ~) and verify that Git is installed:

``` Bash
  git --version
```

3. Configure Your Git Identity

- Set your global username and email address. Important: Use the same email address linked to your GitHub account so your commits are properly attributed to you.

``` Bash
  # Set your full name or GitHub username
  git config --global user.name "Your Name"

  # Set your GitHub account email
  git config --global user.email "your.email@example.com"
```

4. Verify Configuration

- Confirm that your Git credentials were set up correctly:

```Bash
  git config --list
```

(Look for the user.name and user.email entries in the output list.)

Step 7: Synchronize & Push Repository to GitHub

1. **Prevent Secret Leaks:** Verify that environment configuration files (such as .env containing API credentials) and build artifacts are excluded from source control prior to committing.
2. **Initialize Terminal:** Open the integrated terminal in Antigravity IDE (**Ctrl + ~** or **Terminal → New Terminal**).
3. **Configure Git Ignore Rules:**

- Prompt Antigravity Agent in chat: *"Create a .gitignore file for this repository and include all temporary files, dependencies, and environment variables that should not be pushed to GitHub." (Skip if a .gitignore already exists).*

4. **Initialize & Stage Local Repository:**

```
# Initialize Git in the project root
git init

# Stage all codebase files
git add .

# Create initial commit
git commit -m "<Your_commit_message>"
```

5. **Create Remote Repository on GitHub:**
   - Navigate to github.com and click **New Repository**.
   - Enter a repository name (e.g., CropPilotAI).
   - **Important:** Do *not* initialize with a README, .gitignore, or license.
   - Copy the remote HTTPS repository URL (e.g., https://github.com/your-username/CropPilotAI.git).
6. **Link Remote & Push Codebase:**
   - Run the following commands in the Antigravity terminal and complete GitHub authentication when prompted:

```
# Rename default branch to main
`git branch -M main`

# Link your local repo to GitHub (replace this URL with your repo URL)
`git remote add origin https://github.com/your-username/your-repo-name.git`

# Push your code to GitHub
`git push -u origin main`
```

Step 8: Publish & View Your Live Application

Deploy Backend (Render):

1. **Access Render:** Navigate to render.com and sign in (or create an account using your GitHub credentials).
2. **Create Web Service:** Click **+ New** in the dashboard and select Web Service.

3. **Connect Repository:** Authorize your GitHub account, locate your project repository (e.g., CropPilotAI), and select it.
4. **Configure Deployment Settings:**
    - **Branch**: main
    - **Root Directory:** server
    - **Build Command:** npm install
    - **Start Command:** npm start
    - **Instance Type:** Select Free (or your preferred instance plan)
5. **Inject Environment Variables:**
    - Scroll to the Environment Variables section.
    - Add all key-value pairs configured in your local `server/.env` file (e.g., GEMINI_API_KEY, JWT_SECRET, and Supabase credentials)
6. Trigger Deployment: Scroll to the bottom of the configuration page and click **Deploy Web Service**. Render will compile, build, and deploy your live backend API.

Deploy Frontend (Vercel):

1. Access Vercel & Authenticate: Navigate to vercel.com, click Log In / Sign Up, and select Continue with GitHub under the Import Git Repository section. Click on Install to install the Vercel GitHub Application and complete account email verification if prompted.
2. Import Project Repository:
    - First-Time Users: Click Import next to your project repository (e.g., CropPilotAI).
    - Existing Users: Click Add New in the top-right corner, select Project, and click Import next to your target repository.
3. Configure Project & Environment:
    - Assign a project name and set the framework preset to Vite under the Application Preset dropdown.
    - Click Edit next to Root Directory and select your frontend directory (client).
    - In the Environment Variables section, add all variables from your client/.env file.
    - Critical: Replace any localhost backend URLs with your live Render backend URL (ensuring you append /api if your API routing requires it).
4. Deploy Application: Click Deploy. Vercel will build and serve your static React bundle.
5. Troubleshoot Build Errors (If Applicable):
    - Ensure the Root Directory is explicitly set to client.
    - Confirm the frontend code does not directly import files or modules from the /server directory (Vercel only installs packages listed in client/package.json).
6. Configure Backend CORS Settings:
    - Return to the Render Dashboard and open your deployed backend service.
    - Navigate to Environment in the left sidebar.
    - Update the CLIENT_URL key with your live Vercel production URL.
    - Save the environment changes and click Manual Deploy --> Deploy latest commit to enforce cross-origin resource sharing (CORS) compatibility.